Programmer Manual

Lost Grenadiers Navigation System

Prepared by: Audrey Dickson, Kaden Langford, Andrew Swartz

March 27, 2026

1. Vision Statement

The Lost Grenadiers Navigation System provides a clear and accessible way to navigate campus for students, faculty, and visitors. It supports both indoor and outdoor routing using real-time location data, with the option of accessibility consideration in route selection.

2. Introduction

This manual documents the Lost Grenadiers Navigation System for developers. It explains the system architecture, components, tools, and deployment so the system can be maintained and extended upon.

This document serves as a reference for future development. Read the entire document *before* jumping into it.

3. Component Overview

3.1 Frontend (Leaflet)

The frontend component is implemented using Leaflet.js and runs entirely within the client's web browser. It is responsible for rendering the interactive map interface and facilitating user interaction.

The frontend communicates with the backend API via HTTP requests using JSON data.

Responsibilities:

- Rendering the campus map and overlays
- Capturing and processing user input
- Displaying computed routes visually on the map

3.2 Reverse Proxy Layer (Nginx and Apache)

The reverse proxy layer is responsible for handling external traffic, enforcing security, and routing requests to internal services.

Nginx operates at the network edge and manages secure external communication.

Responsibilities:

- Handling HTTPS connections using Let's Encrypt certificates
- Terminating SSL/TLS encryption

- Forwarding incoming requests to internal services

Apache is deployed within the DMZ and serves as an intermediary between the reverse proxy and the application server.

Responsibilities:

- Receiving proxied traffic from Nginx
- Operating within a segmented DMZ
- Forwarding application requests to Gunicorn server

3.3 Application Server

The application server layer is responsible for executing backend logic and serving API responses. All routing logic is executed in the database layer, with Flask acting as a thin API layer.

Gunicorn is used as the WSGI server to run the Flask application.

Responsibilities:

- Hosting the Flask application
- Managing worker processes for concurrent request handling

Flask provides the backend API and handles all application logic.

Responsibilities:

- Defining and exposing API endpoints
- Processing routing and search queries
- Validating user input
- Returning structured JSON responses

Endpoint: *GET /api/get-full-route*

Description: Generates a complete navigation route from a user's current location to a specified campus room. Supports both indoor and outdoor routing, as well as accessibility modes.

Parameters:

- *lat* (float): User latitude
- *lon* (float): User longitude

- *room* (string): Destination room identifier
- *accessible* (boolean): Enables accessibility mode

3.4 Database Layer

The database layer is responsible for storing spatial data and performing routing computations via:

- PostgreSQL
- PostGIS
- pgRouting

Responsibilities:

- Storing graph data (nodes and edges)
- Executing Dijkstra's algorithm
- Maintaining spatial indexing
- Returning route coordinate data to the application layer

3.5 Infrastructure (Proxmox)

The system is hosted on a virtualized environment using Proxmox VE.

Responsibilities:

- Hosting the virtual machine running the application stack
- Providing system isolation and resource allocation
- Support snapshots and backups
- Manage network segmentation (DMZ)
- Enabling high availability

4.0 Tool Overview

QGIS is used to create and edit spatial data, including: nodes, edges, and building layouts. It is also used to validate geometry and prepare data for database import. The version of this application installed is 3x.x. QGIS also has additional libraries installed:

- GDAL: 3.x.x - Handles raster/vector data I/O and geometry handling.
- PROJ: 9.x.x - Controls coordinate transformations (EPSG:4326)
- GEOS: 3.x.x - Geometry engine for spatially correct graph topology
- PostgreSQL client: 17.x

PostgreSQL is the primary database system. It stores all application data, including nodes, edges, and building data. The version installed is 17.x.

- PgRouting is used to perform shortest-path calculations directly within the database. It operates on graph data stored in PostgreSQL. The version installed is 3.x.x.
- DBeaver is used to handle SQL databases + tables through a GUI. The version installed is 26.0.0.2026x

Python 3 is the primary runtime environment for the backend handling of the Flask API and server-side logic. All backend dependencies are installed and run with a Python virtual environment to isolate the project from system-wide packages.

- Flask is the backend framework used to define API endpoints and handle routing logic, as mentioned earlier. The only endpoint is /api/get-full-route. The version installed is 3.1.2.
- Gunicorn is a WSGI server used to run Flask in production and handle incoming HTTP requests. The version installed is 25.x.x.
- Psycopg2 is the database driver for Python we use. The version is 2.x.x.

5.0 Project Repository

5.1 Repository Overview

The Lost Grenadiers Navigation System source code is maintained in a centralized GitHub repository: https://github.com/audreyd114/GitErDone-LostGrenadiers-P445-Capstone

The repository contains all components required to develop, test, and deploy the system. We follow a structure that separates application logic, data, and testing to improve maintainability.

5.2 Software

The primary application code is in the root directory

- app.py
  - The main Flask application file. Defines API endpoints, handles routing logic, and communicates with the database
- index.html
  - Basis for frontend interface
- /css
  - Contains styling and UI customization for web UI
- /js
  - Contains frontend logic, including map interaction
- /indoor
  - Contains indoor mapping data such as floor layouts and overlays

5.3 Test Cases

Testing resources are organized into dedicated directories.

- /cypress
    - Contains frontend tests using Cypress
- /tests
    - Includes additional tests focused on security, API validation, and backend functionality
- cypress.config.js
    - Configuration file for Cypress tests

5.4 Testing Platform

The application is deployed and tested within a virtualized environment via Proxmox VE.

The key characteristics of the setup include:

- A VM hosting the Flask application and PostgreSQL database
- Network segmentation between LAN and DMZ
- Reverse proxy configuration using Nginx and Apache
- HTTPS enabled via Let's Encrypt TLS certificates

This setup allows for realistic testing of security, networking, and performance in a production-like environment.

5.5 Documentation

Documentation is included within the repository and this manual. It describes the system architecture, database structure, and deployment procedures for future developers.

5.4 Test Scripts

Automated and manual test scripts are included in the /tests and /cypress directories. These scripts validate API functionality, frontend behavior, and system security.

6.0 Installation (New System)

From scratch, the installation procedure would be:

1. Install core packages (as listed in the tools subsection)
2. Install pgRouting
3. Configure PostgreSQL
    a. Create the database lostgrenadiers.
    b. Create a user lg_app in the database. This user will be able to only view tables and execute functions on the data. We use this instead of a user with full administrator privileges in the event of SQL injection.
    c. Enable extensions postgis and pgrouting
    d. Import the database schema + data using the full_dump.sql file from the project repository

    e. Importantly, give permissions to the user lg_app the ability to execute the get_full_route() function & the ability to view every table. Do this with the following pseudocode: grant lg_app SELECT privileges ON required_tables AND EXECUTE ON get_full_route().
4. Setup backend Flask and Gunicorn
    a. Clone the git repository.
    b. Create a python venv and within it install flask, psycopg2-binary, and gunicorn.
    c. Gunicorn is bound to 127.0.0.1:5000 and is not exposed publicly. All external traffic is routed through Apache.
5. Configure the app
    a. Update the database connection in app.py with the credentials you created for lg_app.
6. Test server functionality
    a. Execute it with venv/bin/gunicorn -b 127.0.0.1:5000 app:app
    b. Also do a curl with curl http://127.0.0.1:5000/api/health. A valid response should return JSON confirming the API is operational.
7. Startup Services
    a. Automatic Startup: The Gunicorn application server is managed by systemd. A service file starts the backend on boot and restarts it automatically if it exits unexpectedly.
    b. Automatic Repository Updates: A systemd timer is used to periodically fetch the latest code from the GitHub repository and restart the Gunicorn service. This keeps the deployed copy synchronized with the repository without requiring manual intervention. This is technically optional but *wildly* convenient.
8. Configure Apache
9. Enable Apache proxy modules (proxy and proxy_http).
    a. ProxyPreserveHost On
    b. ProxyPass /api/ http://127.0.0.1:5000/
    c. ProxyPassReverse /api/ http://127.0.0.1:5000/
    d. Vitally, ensure Apache's DocumentRoot points to /var/www/lostgrenadiers.
    e. Restart or reload Apache to apply configuration changes.
10. Frontend Deployment
    a. Place frontend files directly from the repo directory at /var/www/lostgrenadiers.
11. The Domain
    a. We purchased the domain lostgrenadiers.org. If you purchase a different domain, you need to set up an A name in your registrar to point to the front facing IP address of the router of whomever is hosting the website.
    b. On the LAN of the person hosting the web server, you'd need to configure the router to enable port forwarding to the host of the web server.
    c. Optionally:

        i. It is highly recommended you harden the security of the network that is hosting the website. You should use a DMZ server for this. Minimally, you should be using a virtual machine to host everything. Do not do this on your personal computer that you use for school work.
        ii. We configured DDNS, or Dynamic DNS. This is because typically internet service providers do not give you a permanent IPv4 address, they can regularly be changed at any time. Dynamic DNS just ensures any time your public IPv4 address changes that the A name in your registrar also updates its answer. You'd do this utilizing the API of your registrar, in our case it was name.com.
12. Other Tips
    a. Use HTTPS for any public deployment. This can be configured using Let's Encrypt and Certbot with Nginx or Apache.
    b. Regular database backups should be performed using pg_dump to prevent data loss. Backups should be stored outside the host system when possible.

7.0 Installation (Migration)

When deploying the system to a new platform using the provided .ova virtual machine image, the installation process is significantly simplified. The .ova contains a preconfigured environment with the application stack, database, and required services already installed.

1. Import the .ova file into a virtualization platform such as Proxmox or VirtualBox.
2. Start the virtual machine.
3. Verify that core services are running, including PostgreSQL, Apache, and Gunicorn.
4. Confirm the backend API is operational by executing:
    a. curl http://127.0.0.1:5000/api/heatlh
5. Update network configuration as necessary:
    a. Assign a new static IP address or DHCP reservation.
    b. Ensure port forwarding is configured if the system is hosted behind a router.
6. Update domain configuration:
    a. Point the domain A record to the new public IP address.
    b. Verify DNS propagation.
7. Validate full system functionality by accessing the frontend and testing route generation.

Because the database is included in the virtual machine image, all existing data is preserved. No additional database import or schema setup is required.

8.0 Further Development

If given more time to contribute to the project, there are several features and QoL aspects that could be implemented. Among these could include:

- A designated mobile app for routing
- An official plugin to be hosted on the one.idu.edu website, which includes authentication and personalization
- The route dynamically updating per time interval based on the current user location
- A student authentication that could grab schedules and route to enrolled classes automatically
- Implementation of external map API to allow for routes beginning off-campus.
- Containerization of servers and systems to ease setup
- Real-time position tracking and re-routing to dynamically update routes as the user moves.
- Multi-building graph refactoring to better support physically connected buildings. This would remove reliance on nearest-entrance heuristics and allow seamless cross-building navigation.
- Implement logging and monitoring to detect failures and performance issues
- Implement the ability for end-users to re-route to another entrance of their building under the condition that the entrance they are routed to is physically blocked off due to external forces such as construction
- Implement the ability for end-users to report a path as “impeded” which would notify the administrator side to temporarily disable routing along said route, preventing future students from being routed to the same location.